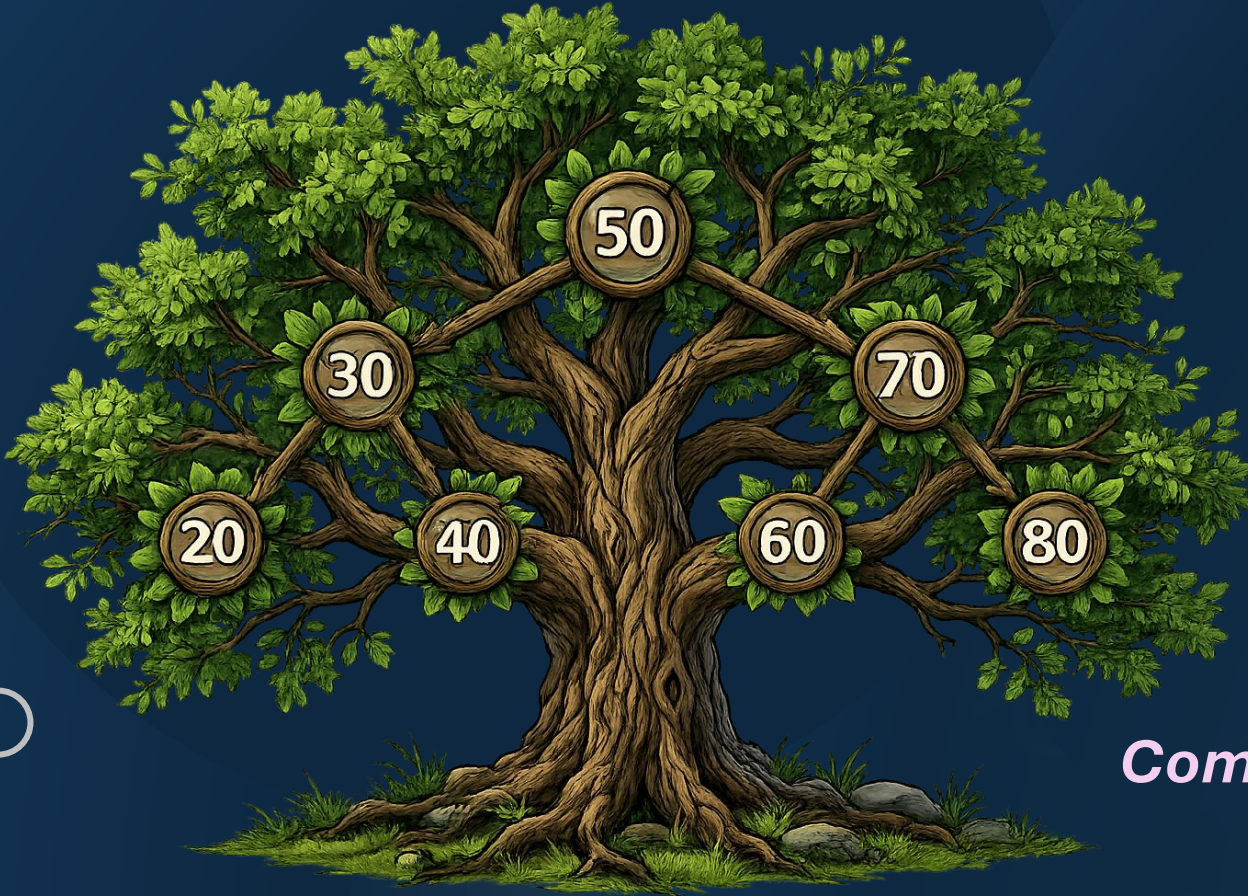




Árboles

INTRODUCCIÓN A ÁRBOLES Y ABB

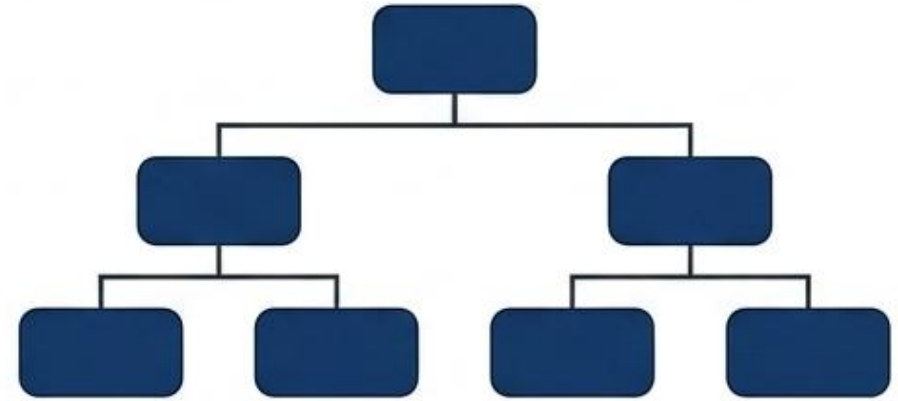
*Comprender estructuras jerárquicas y su
potencia para búsqueda ordenada.*



De estructuras lineales a estructuras jerárquicas



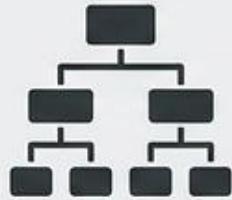
Secuencia única. Un elemento detrás del otro
(Pila, Cola, Lista).



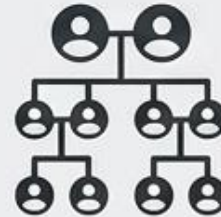
Ramificación y dependencias. Un elemento
contiene o dirige a múltiples elementos.



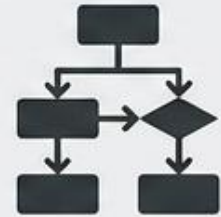
Carpetas de PC



Organigrama



Árbol Genealógico

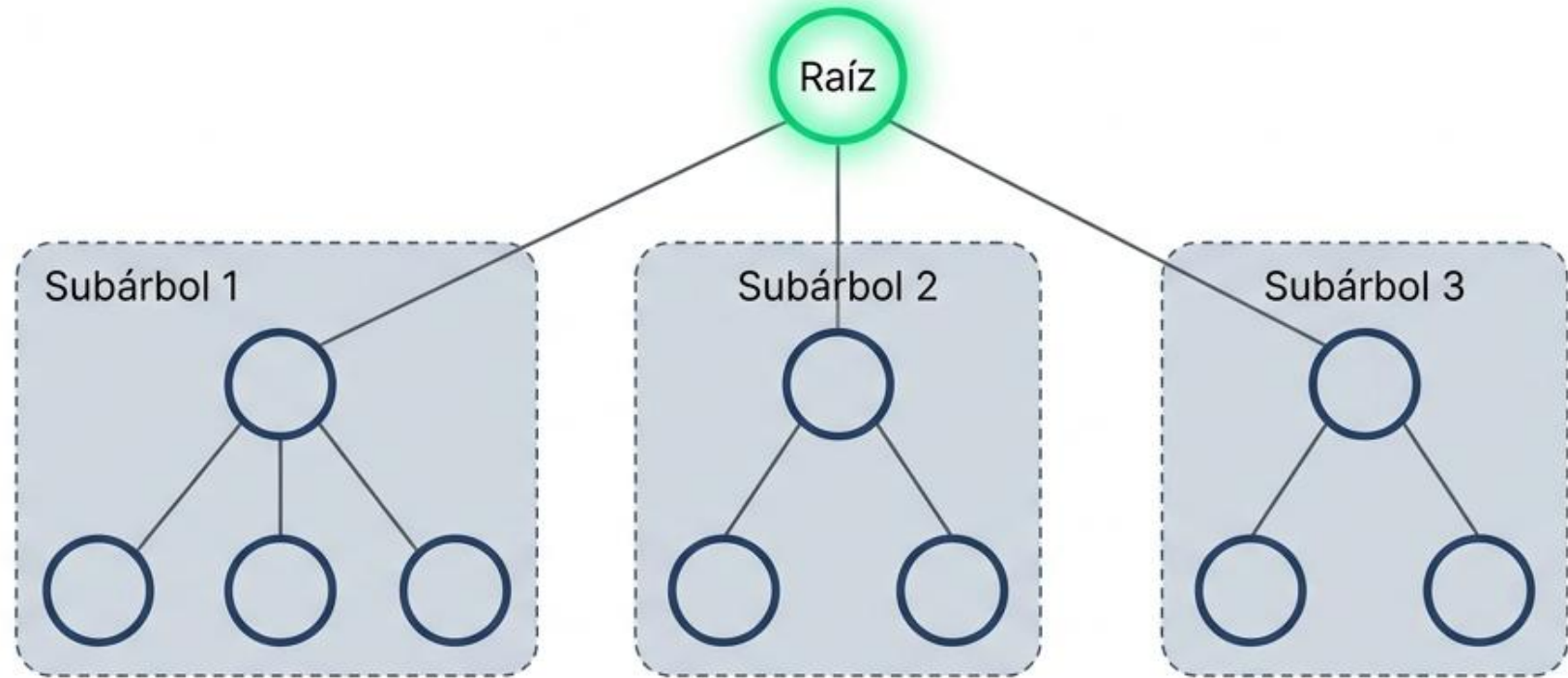


Árbol de Decisiones



Concepto de árbol: Anatomía y Recursividad

Estructura de datos no lineal formada por nodos. Todo nodo tiene un único padre, excepto un nodo especial llamado raíz, que no tiene padre.

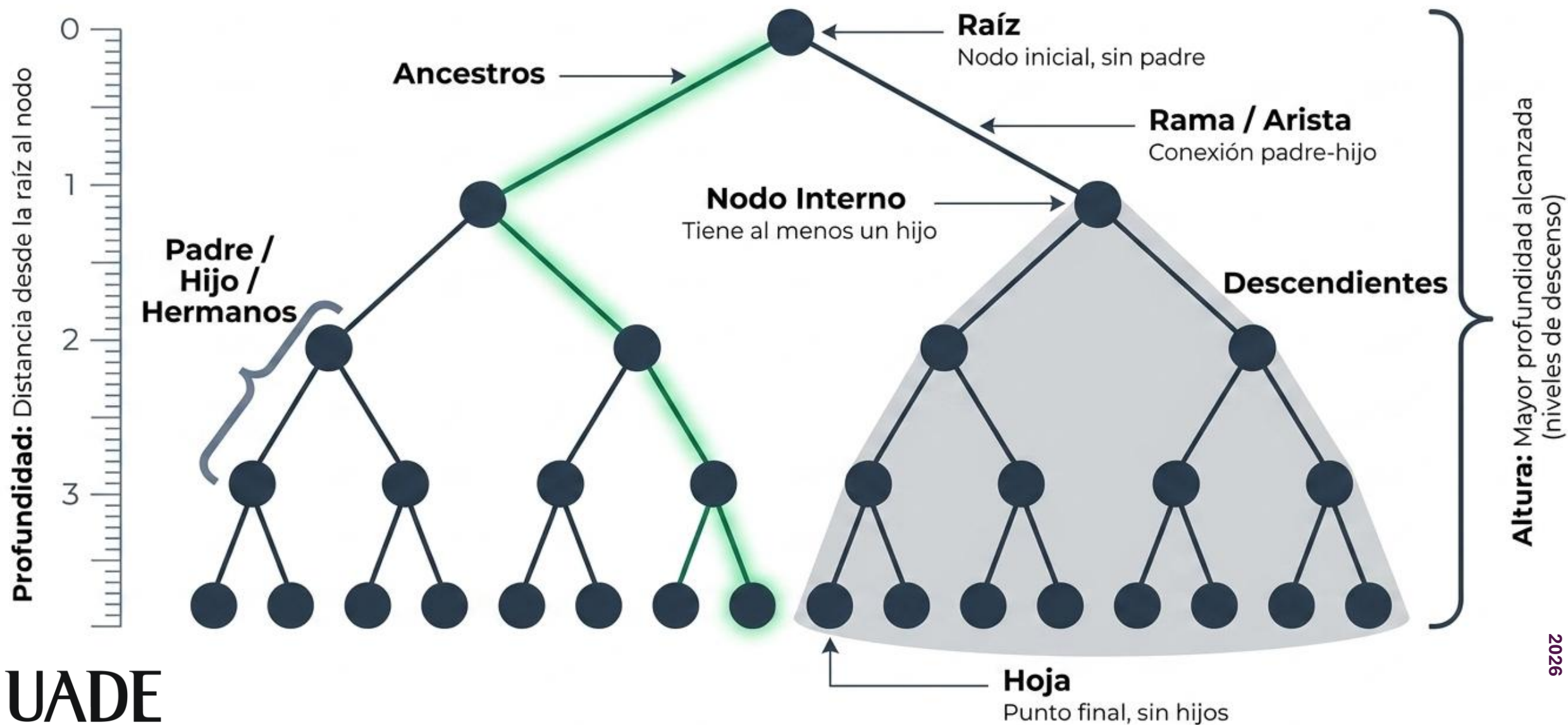


“Un árbol puede estar vacío, o estar formado por una RAÍZ y un conjunto de SUBÁRBOLES.”

Esta definición matemática es la clave para programar.

Todo problema en un árbol se resuelve procesando la raíz y delegando el mismo problema a sus subárboles.

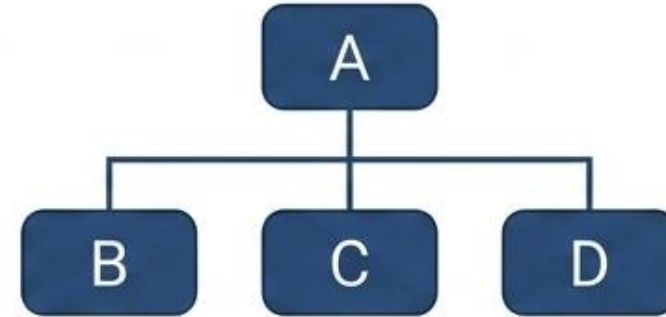
Componentes principales de un árbol



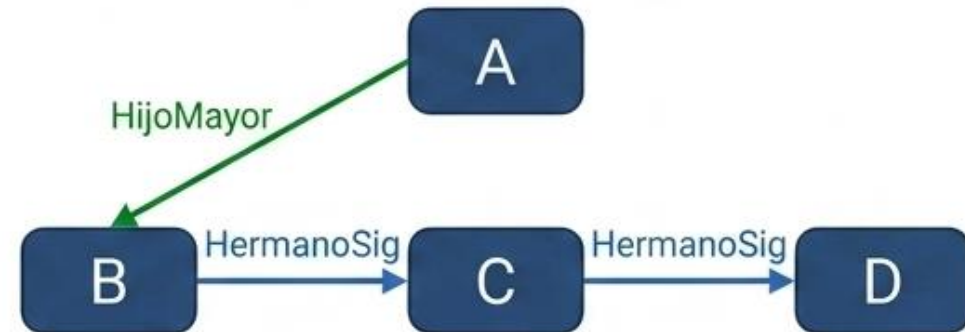
TDA Árbol: Contrato y Abstracción

Contrato ANaTDA

```
Valor()  
ArbolVacio()  
InicializarArbol()  
CrearArbol()  
HijoMayor()  
HermanoSig()  
AgregarHijoM()  
ElimHijoMConDesc()  
ElimHermSConDesc()
```



Transformado
mediante TDA...

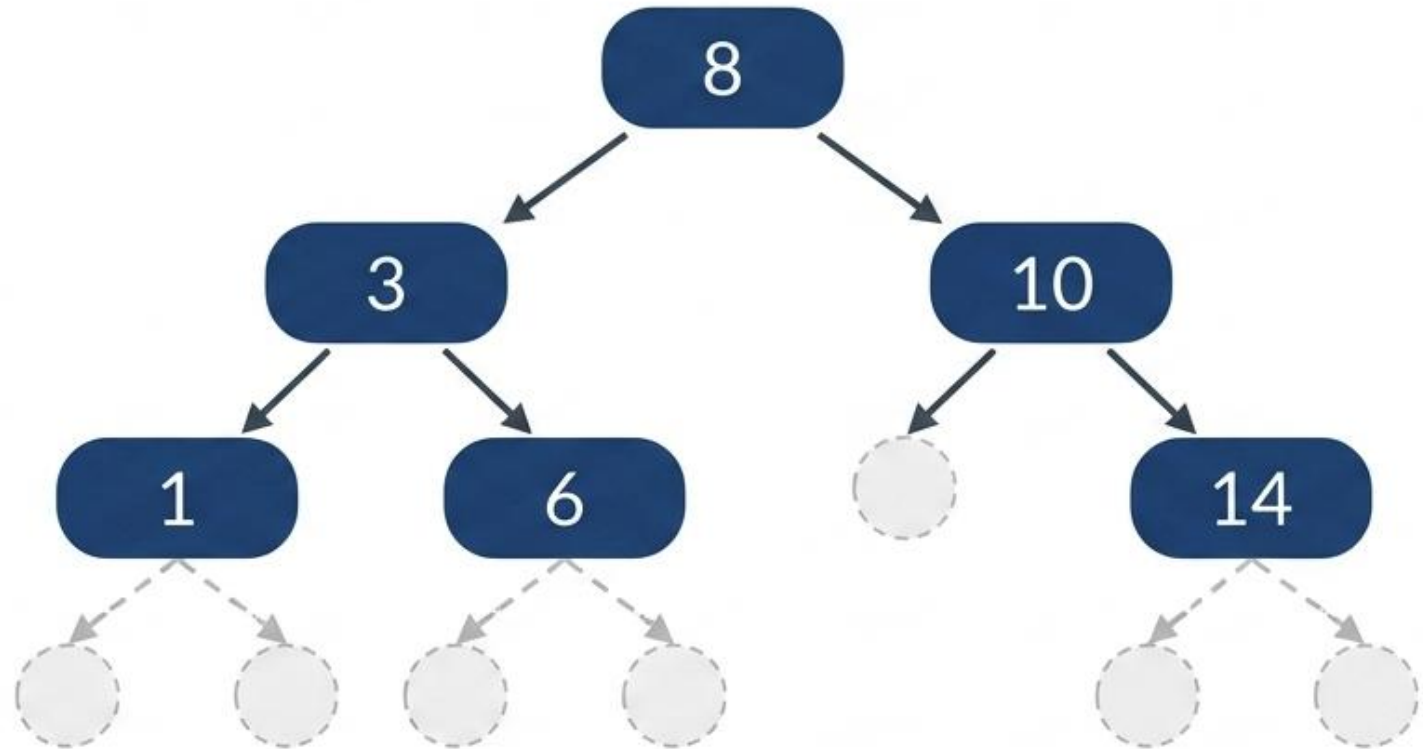


Esta técnica permite representar árboles n-arios (con cualquier cantidad de hijos) en memoria, guardando solo una referencia al primer hijo y otra al siguiente hermano.



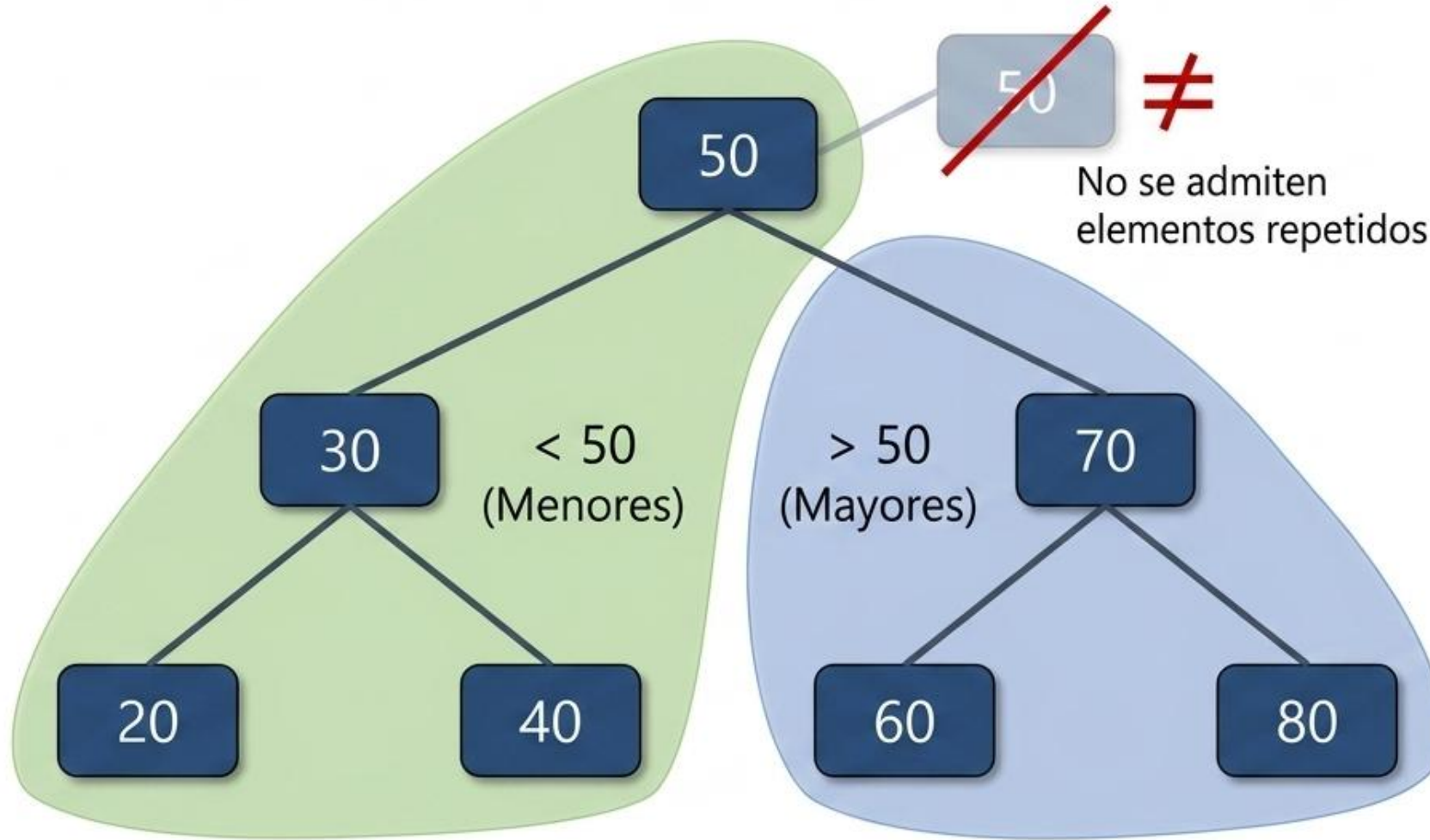
Árbol Binario: La restricción estructural

Un caso particular donde cada nodo tiene como máximo dos hijos: hijo izquierdo e hijo derecho (0, 1 o 2 hijos).



Atención: Un árbol binario NO está necesariamente ordenado. La restricción es puramente de cantidad, no de valor.

Árbol Binario de Búsqueda (ABB)



Contrato ABBTDA

Raiz()

HijoIzq()

HijoDer()

ArbolVacio()

InicializarArbol()

AgregarElem(x)

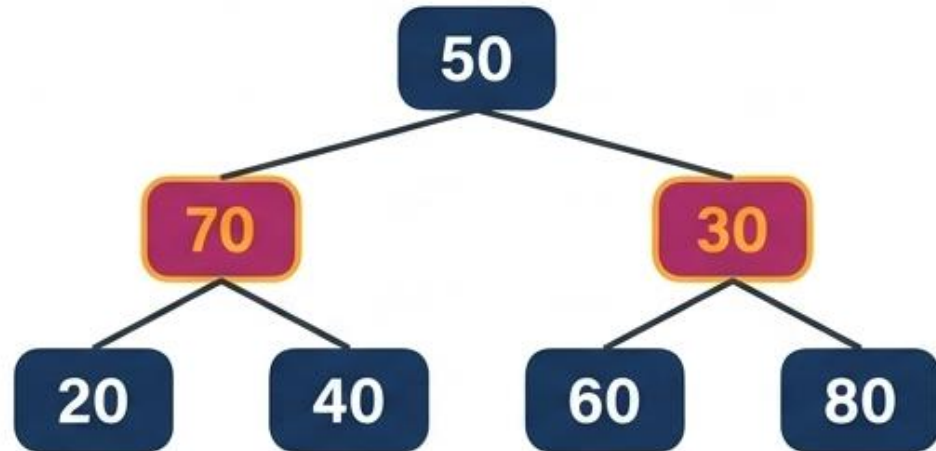
EliminarElem(x)

Conocido en la literatura como BST (Binary Search Tree). En la materia utilizaremos la sigla ABB.

2026

Árbol Binario vs. ABB: Diagnóstico

Árbol Binario (Sin orden)



Árbol Binario (Sin orden). Falla la regla: 70 no es menor que 50.

ABB Correcto

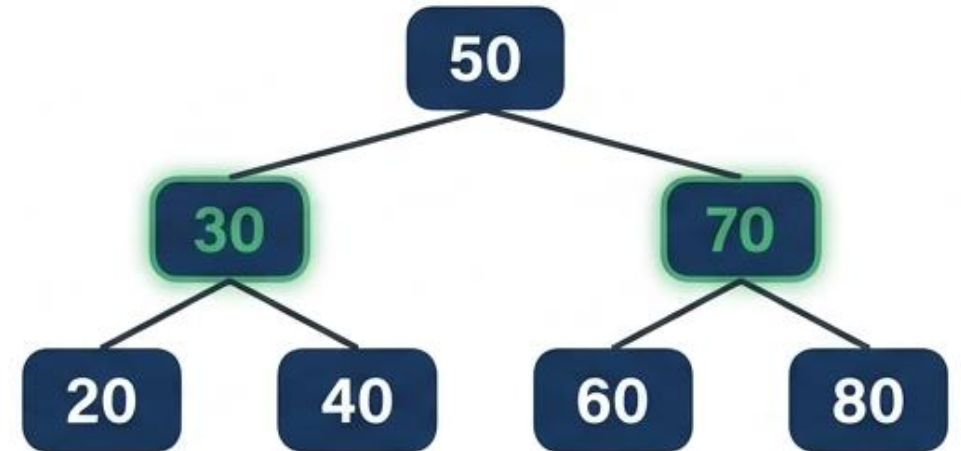
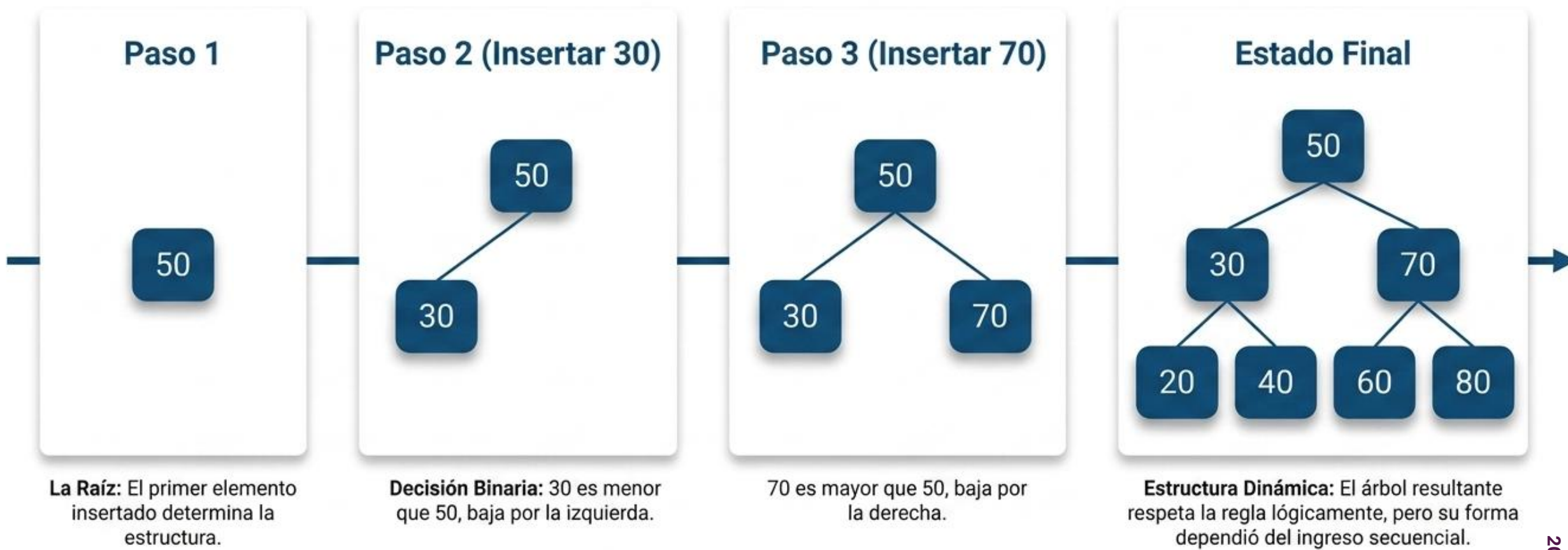


ABB Correcto. Subárbol izquierdo estrictamente menor, subárbol derecho estrictamente mayor.

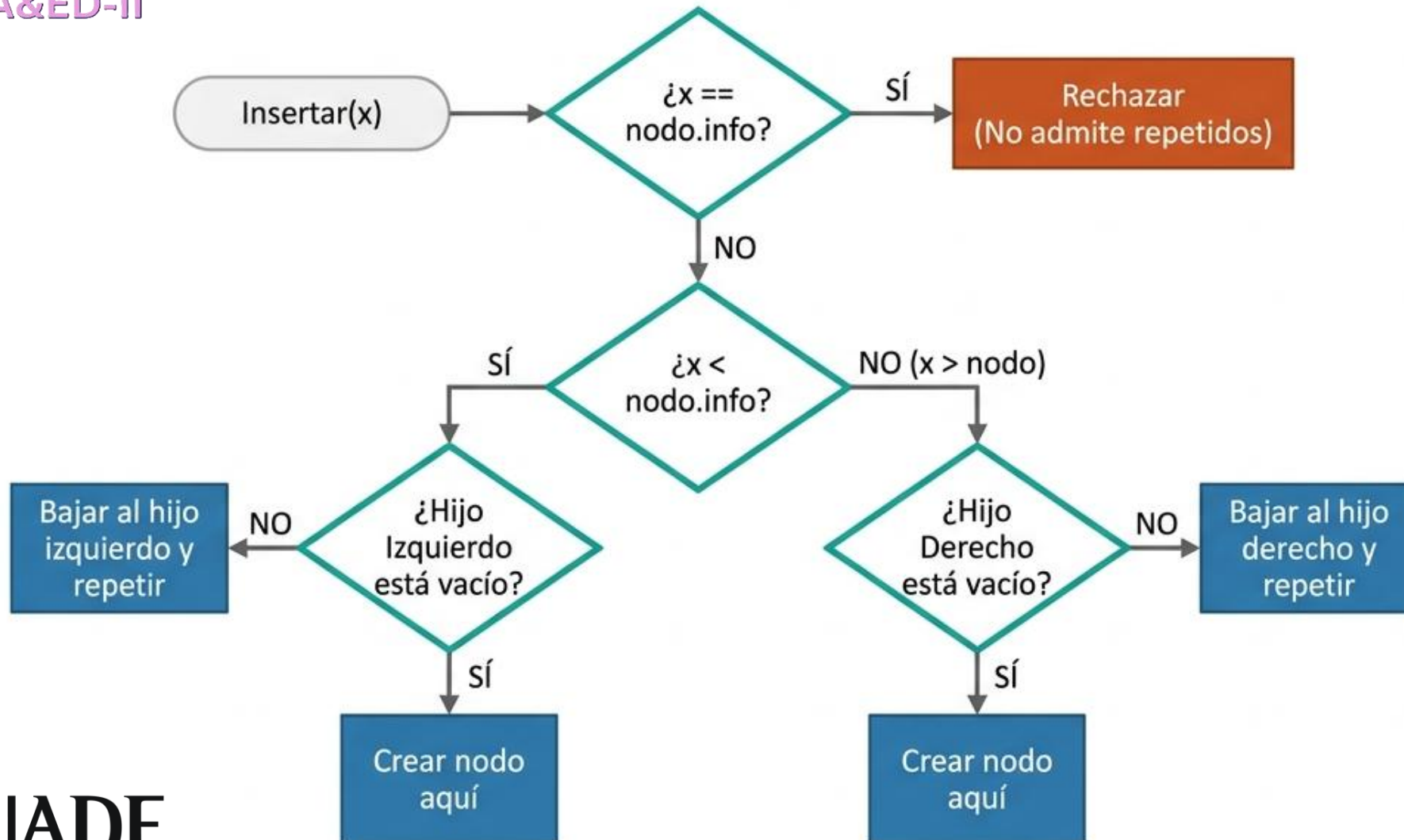
¿Qué ventaja tiene esta regla de orden al momento de buscar un valor frente a una lista enlazada?

Construcción manual de un ABB

Aterrizando el concepto visualmente: la forma del árbol depende del orden de llegada de los datos.



Regla de decisión e inserción en ABB



Aclaración Semántica

- No es FIFO ni LIFO.
- La posición de inserción es estrictamente semántica (por comparación de valor iterativa), no cronológica ni apilada.

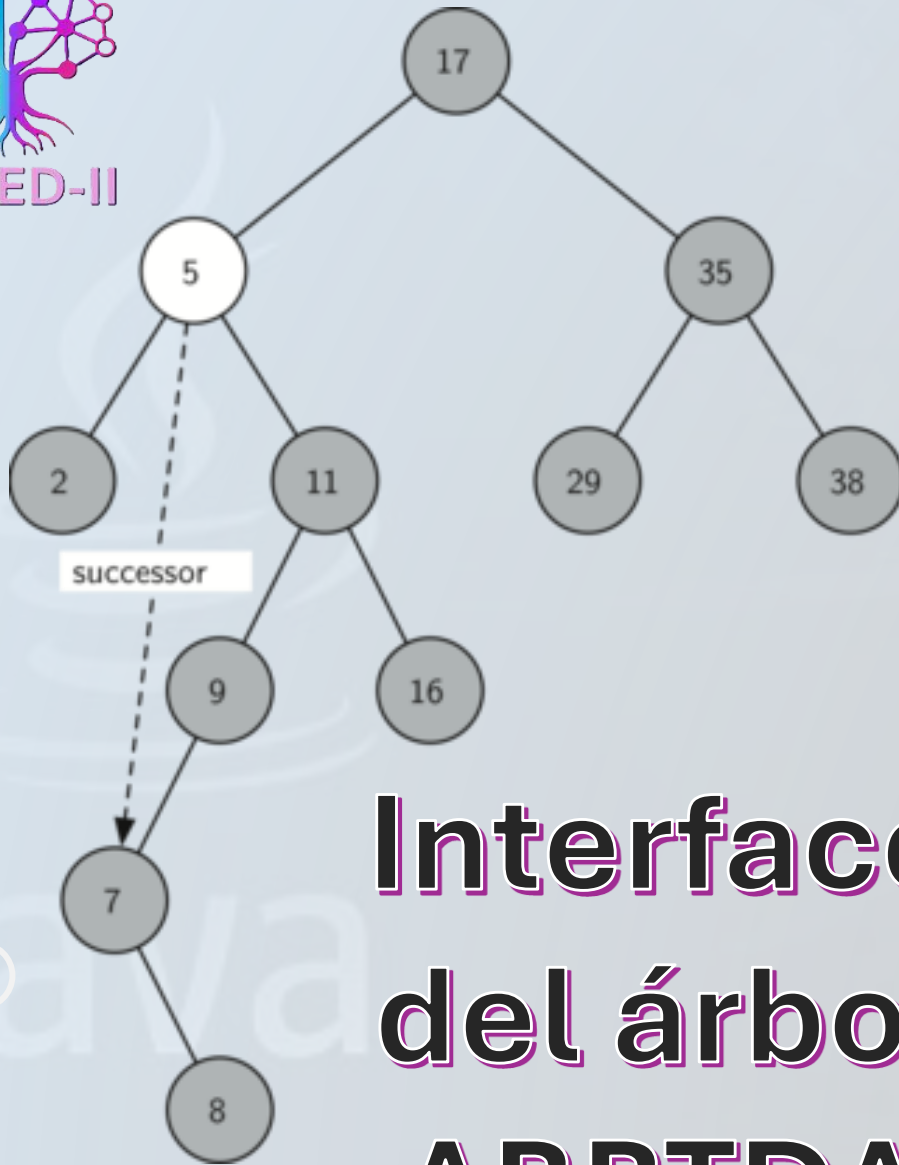
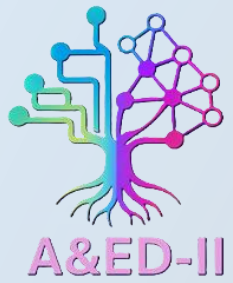
Especificación del TDA ABB: El Contrato

El plano arquitectónico que define las operaciones permitidas antes de escribir la lógica.

Operación	Firma	Precondiciones
InicializarArbol()	void	Ninguna.
ArbolVacio()	boolean	Árbol inicializado.
Raiz() 	int	Árbol inicializado y NO vacío.
Hijolzq() / HijoDer() 	ABBTDA	Árbol inicializado y NO vacío.
AgregarElem(x)	void	Árbol inicializado. Mantiene la propiedad de orden.
EliminarElem(x)	void	Árbol inicializado. Mantiene la propiedad de orden. (Se desarrolla en clases futuras).



Bloqueado: lanza error si se invoca sobre un árbol vacío.



Interface del árbol ABBTDA

UADE

```
public interface ABBTDA {
```

```
// Precondición: el árbol está inicializado y no está vacío.  
int Raiz();
```

```
// Precondición: el árbol está inicializado y no está vacío.  
ABBTDA HijoIzq();
```

```
// Precondición: el árbol está inicializado y no está vacío.  
ABBTDA HijoDer();
```

```
// Precondición: el árbol está inicializado.  
boolean ArbolVacio();
```

```
// Inicializa el árbol como árbol vacío.  
void InicializarArbol();
```

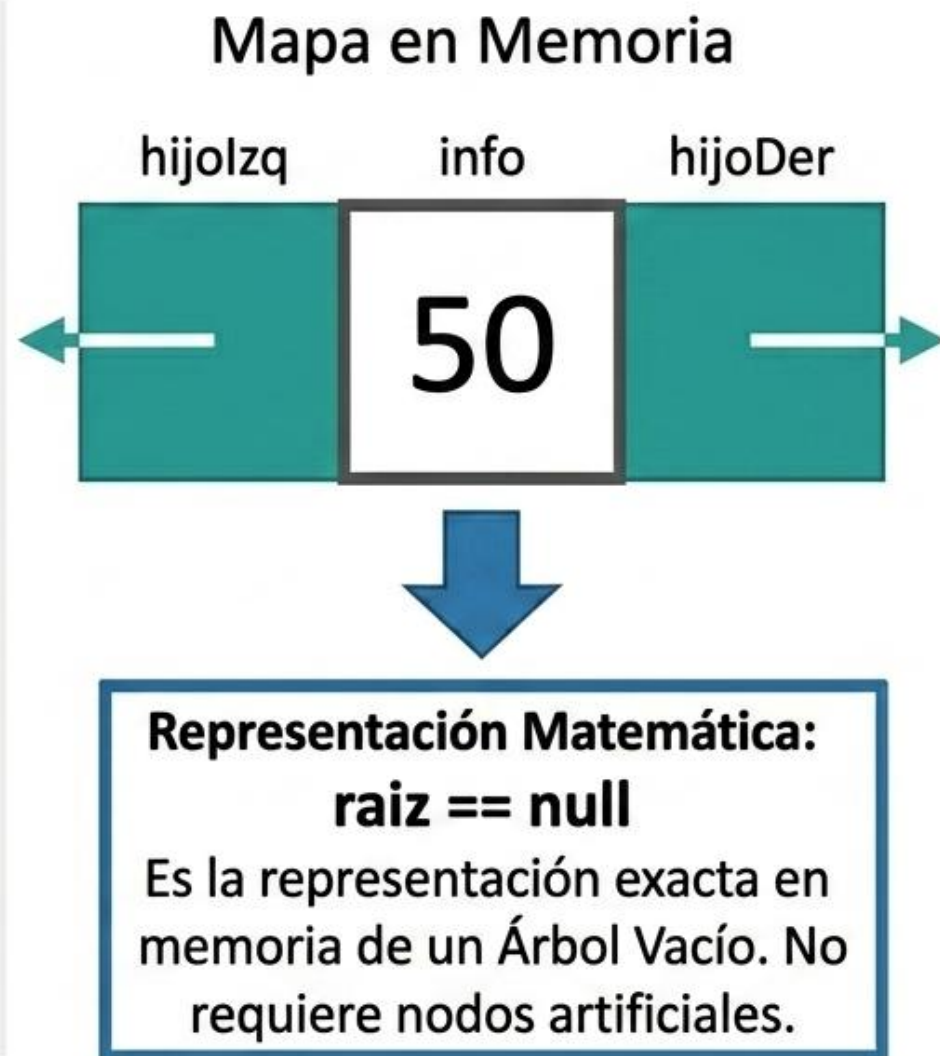
```
// Precondición: el árbol está inicializado.  
// Agrega x manteniendo la propiedad de ABB.  
// Si x ya existe, no se agrega.  
void AgregarElem(int x);
```

```
// Precondición: el árbol está inicializado.  
// Elimina x, si existe, manteniendo la propiedad de ABB.  
void EliminarElem(int x);
```

```
}
```

Implementación dinámica básica del ABB

```
class NodoABB {  
    int info;  
    ABBTDA hijoIzq;  
    ABBTDA hijoDer;  
}  
  
public class ABB implements ABBTDA {  
    NodoABB raiz;  
  
    public void InicializarArbol() { raiz = null; }  
  
    public boolean ArbolVacio() { return raiz == null; }  
  
    public int Raiz() { return raiz.info; }  
  
    // HijoIzq y HijoDer retornan raiz.hijoIzq y raiz.hijoDer  
}
```





Raiz() ArbolVacio() InicializarArbol()

```
class NodoABB {  
    int info;  
    ABBTDA hijoIzq;  
    ABBTDA hijoDer;  
}
```

- `InicializarArbol()` deja el árbol vacío poniendo la raíz en `null`.
- `ArbolVacio()` consulta si la raíz sigue siendo `null`.
- `Raiz()` devuelve el valor guardado en el nodo raíz, pero solo debe usarse si el árbol no está vacío, porque si `raiz == null` no hay valor para devolver.

```
public class ABB implements ABBTDA {  
  
    private NodoABB raiz;  
  
    @Override  
    public void InicializarArbol() {  
        raiz = null;  
    }  
  
    @Override  
    public boolean ArbolVacio() {  
        return raiz == null;  
    }  
  
    @Override  
    public int Raiz() {  
        return raiz.info;  
    }  
}
```

Implementación de AgregarElem

```
public void AgregarElem(int x) {  
    if (raiz == null) {  
        raiz = new NodoABB();  
        raiz.info = x;  
        raiz.hijoIzq = new ABB();  
        raiz.hijoIzq.InicializarArbol();  
        raiz.hijoDer = new ABB();  
        raiz.hijoDer.InicializarArbol();  
    } else if (x < raiz.info) {  
        raiz.hijoIzq.AgregarElem(x);  
    } else if (x > raiz.info) {  
        raiz.hijoDer.AgregarElem(x);  
    }  
}
```

Caso Base

Llegamos a un árbol vacío.
Se instancia el nodo y se inicializan obligatoriamente sus dos hijos como nuevos ABB vacíos.

Paso Recursivo

El problema se delega idénticamente al subárbol izquierdo o derecho.
No hay ciclos (for/while).

Omisión por igualdad

Si $x == \text{raiz.info}$, el código simplemente termina. El TDA rechaza los elementos repetidos de forma natural.



```
@Override
public ABBTDA HijoIzq() {
    return raiz.hijoIzq;
}

@Override
public ABBTDA HijoDer() {
    return raiz.hijoDer;
}
```

- `HijoIzq()` devuelve el subárbol izquierdo de la raíz actual.
- `HijoDer()` devuelve el subárbol derecho. Ambos métodos tienen como precondition que el árbol no esté vacío, porque si `raiz == null` no habría hijos para consultar.

HijoIzq()

HijoDer()

AgregarElem()

```
@Override
public void AgregarElem(int x) {
    if (raiz == null) {
        raiz = new NodoABB();
        raiz.info = x;

        raiz.hijoIzq = new ABB();
        raiz.hijoIzq.InicializarArbol();

        raiz.hijoDer = new ABB();
        raiz.hijoDer.InicializarArbol();
    } else if (x < raiz.info) {
        raiz.hijoIzq.AgregarElem(x);
    } else if (x > raiz.info) {
        raiz.hijoDer.AgregarElem(x);
    }
}
```



`EliminarElem(x)` primero busca el valor respetando la regla del ABB. Si x es menor que la raíz, baja por izquierda; si es mayor, baja por derecha.

Cuando encuentra el nodo, aparecen tres casos:

- si es hoja, lo elimina poniendo `raiz = null`;
- si tiene un solo hijo, reemplaza la raíz por ese hijo;
- si tiene dos hijos, reemplaza el valor de la raíz por el mayor valor del subárbol izquierdo y luego elimina ese valor duplicado del subárbol izquierdo.

```
@Override
public void EliminarElem(int x) {
    if (raiz != null) {
        if (x < raiz.info) {
            raiz.hijoIzq.EliminarElem(x);

        } else if (x > raiz.info) {
            raiz.hijoDer.EliminarElem(x);

        } else if (raiz.hijoIzq.ArbolVacio() && raiz.hijoDer.ArbolVacio()) {
            raiz = null;

        } else if (raiz.hijoIzq.ArbolVacio()) {
            raiz = ((ABB) raiz.hijoDer).raiz;

        } else if (raiz.hijoDer.ArbolVacio()) {
            raiz = ((ABB) raiz.hijoIzq).raiz;

        } else {
            raiz.info = mayor(raiz.hijoIzq);
            raiz.hijoIzq.EliminarElem(raiz.info);
        }
    }
}

private int mayor(ABBTDA a) {
    if (a.HijoDer().ArbolVacio()) {
        return a.Raiz();
    } else {
        return mayor(a.HijoDer());
    }
}
```

Recursividad estructural en árboles

La estructura de memoria dicta la estructura del algoritmo.

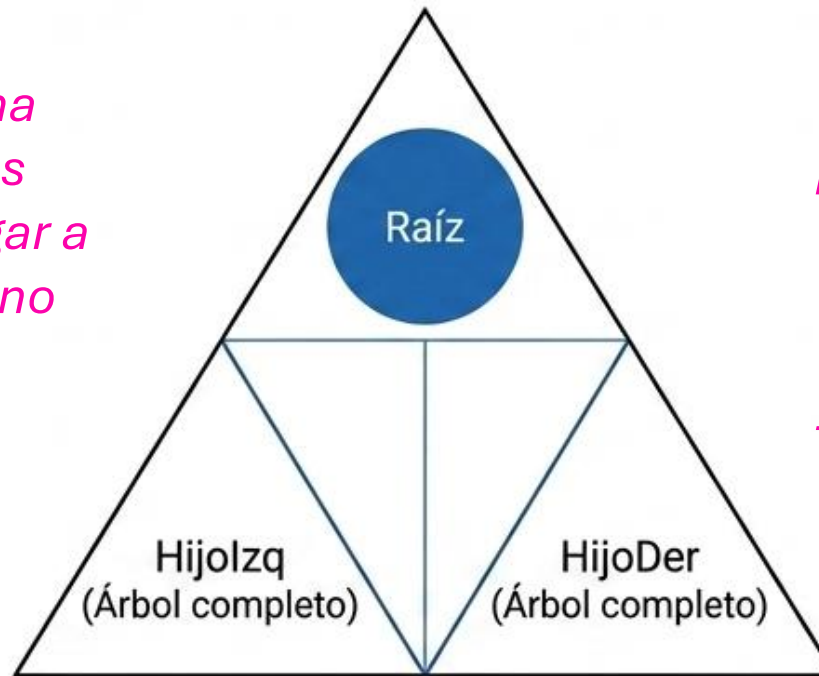
Recordatorio Matemático

$\text{factorial}(n) = n * \text{factorial}(n-1)$

Un método que se invoca a sí mismo sobre un sub-problema más pequeño hasta llegar a un caso base.

$\text{Árbol} = \text{Raíz} + \text{Subárbol Izquierdo} + \text{Subárbol Derecho}$

Recursividad es repetir la misma solución sobre una versión más pequeña del problema, hasta llegar a una situación mínima donde ya no hace falta seguir.



En árboles, cada hijo de un nodo puede verse como otro árbol más pequeño. Entonces el método hace siempre lo mismo:

- si el árbol está vacío, corta;
- si no está vacío, procesa la raíz y vuelve a llamarse sobre el hijo izquierdo y el hijo derecho.

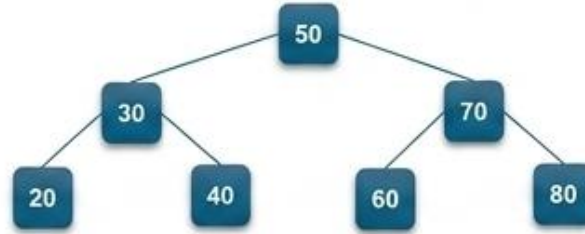
Caso Base

Si el árbol está vacío:
Cortar recursión.

Paso Recursivo

Si no está vacío:
Procesar la raíz, delegar recursivamente al Hijolq y luego al HijoDer.

Recorridos clásicos: Procesando todo el árbol



PREORDEN

Raíz, Izquierda, Derecha.

```
print(a.Raiz());  
pre(a.HijoIzq());  
pre(a.HijoDer());
```

50, 30, 20, 40, 70, 60, 80

INORDEN

Izquierda, Raíz, Derecha.

```
in(a.HijoIzq());  
print(a.Raiz());  
in(a.HijoDer());
```

20, 30, 40, 50, 60, 70, 80

¡Devuelve los valores
ordenados de menor a mayor!

POSTORDEN

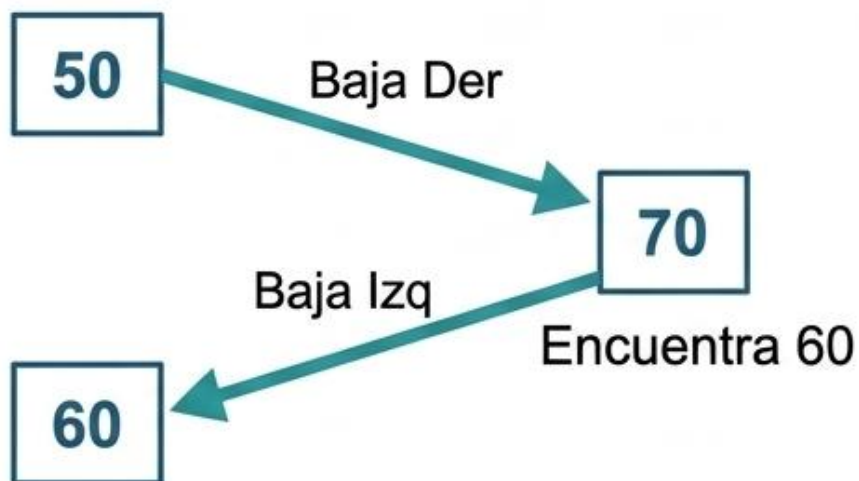
Izquierda, Derecha, Raíz.

```
post(a.HijoIzq());  
post(a.HijoDer());  
print(a.Raiz());
```

20, 40, 30, 60, 80, 70, 50

Búsqueda y una primera mirada al costo

El Algoritmo: `existe(a, x)`



Compara con 50 (Mayor) → Baja Der →
Compara con 70 (Menor) → Baja Izq →
Encuentra 60.

- Retorna true al encontrarlo o false al llegar a un árbol vacío.

El Costo Computacional depende de la altura

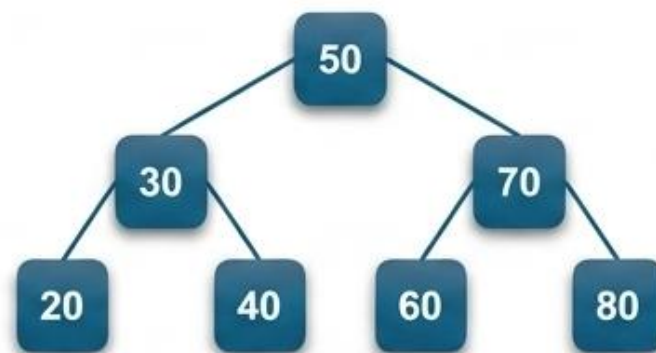


ABB Balanceado.
Bien distribuido.
Pocos niveles.
Búsqueda rápida.

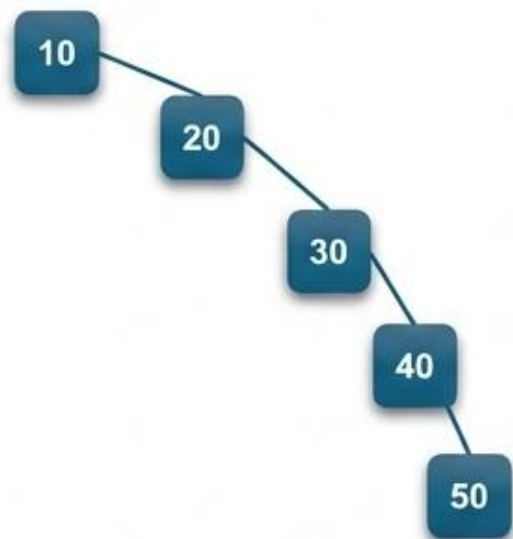
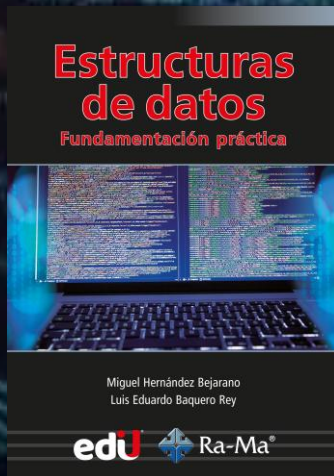
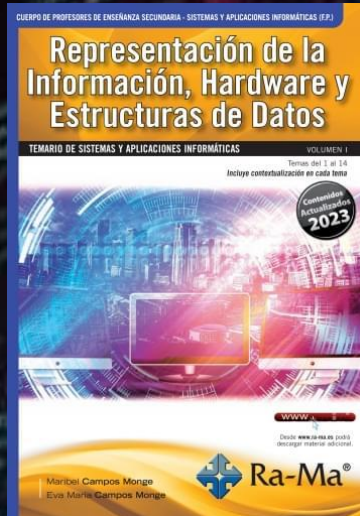


ABB Desbalanceado
(Árbol Degenerado).
Se comporta como
una lista enlazada.
Alto costo, recorre
todos los elementos.



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 9

Resolver problemas con ABB

Altura, nodos hoja, balance conceptual.

Aplicaciones prácticas.